

PQ-Shield — Build Pipeline

Target hardware: ASUS Vivobook 16, Ryzen 7 5800HS, 16GB RAM, CPU-only. This is fine for this project — no GPU work anywhere. The only environment decision that matters is **build liboqs on Linux, not native Windows**.

Phase 0 — Environment Setup (do this before anything else)

0.1 Get a Linux build environment

`liboqs` is a C library built with CMake/Ninja. It builds cleanly on Linux/macOS; on native Windows it's a documented pain (MSVC + vcpkg quirks). Use **WSL2 (Ubuntu 22.04/24.04)**.

```
bash

# In Windows PowerShell (admin)
wsl --install -d Ubuntu-24.04
```

Then do everything below inside WSL2 Ubuntu.

0.2 System dependencies

```
bash

sudo apt update
sudo apt install -y build-essential cmake ninja-build git \
python3 python3-venv python3-pip libssl-dev pkg-config
```

0.3 Build liboqs (C library) from source

```
bash

git clone --depth=1 https://github.com/open-quantum-safe/liboqs.git
cd liboqs
mkdir build && cd build
cmake -GNinja -DBUILD_SHARED_LIBS=ON ..
ninja
sudo ninja install
sudo ldconfig
```

Verify:

```
bash
```

```
ls /usr/local/lib | grep oqs      # should show libliboqs.so
```

0.4 Python environment + bindings

The correct package is `liboqs-python` (PyPI), which you `import oqs`. There is no package literally called `pyoqs` — fix that reference in your own docs/paper.

```
bash
```

```
python3 -m venv ~/pqshield-env
source ~/pqshield-env/bin/activate

pip install --upgrade pip
pip install liboqs-python fastapi uvicorn "cryptography>=42" \
    scikit-learn torch torchvision pandas matplotlib psutil mitmproxy \
    pytest httpx numpy
```

Sanity check:

```
bash
```

```
python3 -c "
import oqs
print('KEMs:', [k for k in oqs.get_enabled_kem_mechanisms() if 'ML-KEM' in k])
print('Sigs:', [s for s in oqs.get_enabled_sig_mechanisms() if 'ML-DSA' in s])
"
```

You should see `ML-KEM-512/768/1024` and `ML-DSA-44/65/87`. If this prints correctly, your entire cryptographic foundation is verified and you can stop worrying about environment issues for the rest of the project.

0.5 Repo layout

```
pq-shield/
├─ model/                # Layer 1
│   ├── train.py
│   └─ artifacts/
├─ api/                  # Layer 2
│   ├── server.py
│   └─ client.py
├─ crypto/               # Layer 3
│   └─ config_a_classical.py
```

```
|   |— config_b_hybrid.py
|   |— config_c_full_pqc.py
|   |— channel.py           # shared handshake/envelope logic
|— threats/                 # Layer 4
|   |— hndl_capture.py
|   |— mitm_addon.py        # mitmproxy script
|— bench/                   # Layer 5
|   |— runner.py
|   |— metrics.py
|— analysis/                # Layer 6
|   |— tradeoff_matrix.py
|   |— figures.py
|— results/
|   |— raw/
|   |— processed/
|— paper/
```

Phase 1 — Weeks 1-3: Foundation

Step 1.1 — Train and freeze the model (`model/train.py`)

- RandomForest on a tabular set, or a small CNN on CIFAR-10 via `torchvision.datasets.CIFAR10`.
- Recommendation given your CPU-only box: **RandomForest on a small tabular dataset** (e.g. sklearn's `load_digits`) or a CIFAR-10 subset with flattened pixels) — trains in seconds, keeps inference latency near-zero so your benchmark numbers reflect *crypto overhead*, not model overhead. If you want the CNN for narrative reasons, train it once, freeze it, and only ever run `model.eval()` inference (no backprop) — a tiny CNN forward pass on CPU is still single-digit milliseconds, which is fine.
- Serialize: `joblib.dump(model, "model/artifacts/model.pkl")` or `torch.save(model.state_dict(), "model/artifacts/model.pt")`.

Step 1.2 — Bare FastAPI endpoint, no crypto (`api/server.py`)

```
python
```

```

from fastapi import FastAPI, Request
import joblib, numpy as np, time

app = FastAPI()
model = joblib.load("model/artifacts/model.pkl")

@app.post("/predict")
async def predict(request: Request):
    body = await request.json()
    x = np.array(body["input"]).reshape(1, -1)
    t0 = time.perf_counter()
    pred = model.predict(x).tolist()
    latency = (time.perf_counter() - t0) * 1000
    return {"prediction": pred, "inference_ms": latency}

```

Run it: `uvicorn api.server:app --port 8000`. Hit it with `curl` or `httpx` and confirm you get predictions back. **This is your control condition with zero crypto overhead** — keep this exact code path unmodified; every configuration wraps around it, never edits it.

Step 1.3 — Write `api/client.py` as a thin `httpx`-based client you'll reuse across all three configs and the benchmark runner.

Phase 2 — Weeks 4-6: Configuration A (Classical Baseline)

Build `crypto/channel.py` first as a shared interface all three configs implement:

```

python

class CryptoChannel:
    def handshake(self): ... # returns (shared_key, handshake_metadata)
    def sign_request(self, payload: bytes) -> bytes: ...
    def verify_response(self, payload: bytes, sig: bytes, pubkey) -> bool: ...

```

Config A (`crypto/config_a_classical.py`) using `cryptography`:

- Key exchange: RSA-2048 — client generates an ephemeral AES session key, encrypts it with the server's RSA-2048 public key (RSA-OAEP), server decrypts. This is the "handshake."
- Signing: ECDSA (P-256) — server signs each response body with its ECDSA private key; client verifies with the server's ECDSA public key.

- Session payloads: encrypt with AES-256-GCM using the exchanged key (this is what a real API does — asymmetric crypto sets up a symmetric session key, then symmetric crypto does the bulk work).

Wrap this around the Phase-1 endpoint via FastAPI middleware or an explicit `/predict` handler variant (`api/server_config_a.py`) that:

1. On startup, generates/loads RSA + ECDSA keypairs.
2. Exposes a `/handshake` endpoint for the client to fetch the RSA public key and perform key exchange.
3. Wraps `/predict` so the body is AES-GCM encrypted, and the response is ECDSA-signed.

Write a round-trip integration test (`pytest`) before moving on: client does handshake, sends encrypted signed request, gets encrypted signed response back, verifies signature, decrypts, matches what the unsecured endpoint returned. **This test is your regression guard for the rest of the project** — Configs B and C should pass the identical test with only the crypto primitives swapped.

Phase 3 — Weeks 7-9: Hybrid and Full PQC

Config B (`crypto/config_b_hybrid.py`):

```
python

import oqs

def handshake_server():
    kem = oqs.KeyEncapsulation("ML-KEM-768")
    pub = kem.generate_keypair()
    return kem, pub    # send pub to client

def handshake_client(server_pub):
    kem = oqs.KeyEncapsulation("ML-KEM-768")
    ciphertext, shared_secret = kem.encap_secret(server_pub)
    return ciphertext, shared_secret    # send ciphertext to server

# server side, after receiving ciphertext:
# shared_secret = kem.decap_secret(ciphertext)
```

- Derive an AES-256-GCM session key from `shared_secret` (HKDF via `cryptography`).
- Signing stays ECDSA — same code as Config A, only the KEM changed.

Config C (`crypto/config_c_full_pqc.py`):

- Same ML-KEM-768 handshake as B.
- Signing swapped to ML-DSA-65:

```
python
```

```
sig = oqs.Signature("ML-DSA-65")
pub = sig.generate_keypair()
signature = sig.sign(message)
oqs.Signature("ML-DSA-65").verify(message, signature, pub)
```

Because you built `channel.py` as a shared interface in Phase 2, B and C should mostly be drop-in swaps of the KEM/signature backend, not new server logic. Re-run the same round-trip pytest against both. By end of week 9 you should have three servers (`server_config_a/b/c.py`) all passing the same test suite, differing only in `crypto/config_*.py`.

Phase 4 — Weeks 10–11: Threat Simulation

HNDL capture (`threats/hndl_capture.py`):

- Script that, for each config, fires 1,000 requests through the client and — instead of discarding the wire data — logs to disk: ciphertext bytes, public key bytes, KEM ciphertext bytes (for B/C), signature bytes, and total per-request byte count.
- Output: a CSV per configuration with columns `config, request_id, ciphertext_bytes, key_bytes, sig_bytes, total_bytes`. This directly answers RQ3.

MITM harness (`threats/mitm_addon.py`), run via `mitmproxy`):

- A mitmproxy addon script that intercepts responses on the loopback proxy and flips a byte in the prediction payload before forwarding it to the client.
- Client-side: measure wall-clock time from receiving the tampered response to the signature-verification call returning `False`/raising. That delta is your MITM detection latency (RQ4).
- Run: `mitmdump -s threats/mitm_addon.py -p 8080`, point your client at the proxy.
- **Validate deliberately:** confirm Config A's ECDSA also rejects tampering (it should — this isn't testing "does PQC catch tampering" vs "does classical not," it's testing *how fast* each catches it). If Config A doesn't reject tampering, your signing wrapper has a bug — fix it before benchmarking.

Phase 5 — Weeks 12–13: Benchmark Runs

`bench/runner.py` — the core loop:

```
python

for config in ["A", "B", "C"]:
    for concurrency in [10, 100, 1000]:
        for threat in ["none", "hndl", "mitm"]:
            for trial in range(5):
                run_benchmark(config, concurrency, threat, trial)
```

- Use `asyncio` + `httpx.AsyncClient` (or `locust`/`hey` as an external load generator if you want less custom code — `hey -n 1000 -c 100 <url>` is a good sanity cross-check against your own harness) to fire concurrent requests.
- Per request, capture: handshake latency, round-trip time, CPU% (`psutil.cpu_percent` sampled during the run), memory (`tracemalloc` snapshots around the crypto calls specifically, not the whole process — otherwise model-loading noise pollutes it).
- **Caveat for your hardware:** 1,000 concurrent requests on an 8-core laptop will contend for CPU heavily regardless of crypto config — that's expected and is itself a finding, but note in your methodology that "concurrency" here means concurrent in-flight requests against a single-process server, not 1,000 real parallel cores. Consider running FastAPI with `uvicorn --workers N` ($N \approx$ number of physical cores, so ~ 8) to get more realistic multi-core behavior, and log worker count in your paper.
- Write raw output to `results/raw/{config}_{concurrency}_{threat}_{trial}.csv`.

`bench/metrics.py` — aggregate: mean, std, 95% CI across the 5 trials per cell.

Phase 6 — Weeks 14–15: Analysis and Visualization

`analysis/tradeoff_matrix.py`:

- Define your composite score explicitly and defensibly — e.g. `score = w1 * normalized_security_strength + w2 * normalized_latency_cost`, where security strength is a categorical→numeric mapping you justify (NIST security levels: ML-KEM-768 $\approx$ Level 3, ML-DSA-65 $\approx$ Level 3, RSA-2048/ECDSA-P256 $\approx$ roughly Level 1 classical equivalent — cite NIST SP 800-208 / FIPS 203/204 for these mappings rather than asserting them).

- Output the matrix as a `pandas.DataFrame`, pivot to (threat × config × concurrency), export to `results/processed/tradeoff_matrix.csv` and a heatmap figure.

`analysis/figures.py`:

- Latency bar chart (3 configs × 3 concurrency levels, grouped bars).
- Ciphertext volume comparison (RQ3) — bar chart, bytes per 1,000 requests.
- CPU/memory heatmaps.
- MITM detection latency comparison (RQ4) — this is probably your most striking chart; lead with it if ML-DSA-65 verification turns out competitive with ECDSA despite being PQC (worth flagging early if you see this trend, since it's counter to naive "PQC = slow" intuition).

At this stage, sit with the numbers for a day before writing conclusions — flag which results are surprising (e.g., is Hybrid actually meaningfully cheaper than Full PQC, or does ECDSA signing dominate the cost either way, making the KEM choice matter more than the signature choice? That would reframe RQ2's answer).

Phase 7 — Week 16: Paper and Repository

- Structure paper around the 4 RQs directly — each RQ gets its own results subsection with the table/figure that answers it.
 - Reproducibility README: exact `liboqs` commit hash, `liboqs-python` version, OS/kernel (note WSL2 if used — reviewers on bare Linux may see slightly different numbers, worth a footnote), hardware spec, and the exact `pip freeze` output committed to the repo.
 - Target: IEEE Access (broader scope, values reproducibility) or a CCS workshop (SafeThings/CPS focused workshops if you want the "protecting an AI system" framing to land more directly).
-

Key risks to watch for early, not at week 15

1. **liboqs build failing on WSL2** — do Phase 0 in isolation, week 1, before writing any project code. If it doesn't work, you have 15 weeks to find a workaround (Docker container with a prebuilt liboqs image is your fallback).
2. **Model inference time swamping crypto overhead** — if your model takes 50ms and your crypto adds 2ms, your benchmark has nothing interesting to say. Keep the model trivially fast (RandomForest on small tabular input) so crypto overhead is the dominant, measurable signal.

3. **mitmproxy + custom TLS/crypto layer conflict** — since you're implementing your own application-layer crypto (not relying on TLS), mitmproxy just needs to sit as a plain HTTP proxy passing your already-encrypted-at-the-app-layer bytes through, then tamper with them. Confirm this works in week 10 before assuming it will.
4. **1,000-concurrency numbers being CPU-bound noise on a single laptop** — plan to caveat this explicitly in the paper rather than over-claim, per point above.